Minimal Implementation of the Algorithmic Life Equation

A Computational Scaffold for Consciousness-Related Functions

Concrete Functional Forms, Thermodynamically Consistent Dynamics, and Experimental Protocol

Companion to "The Thermodynamics of Being" and "The Algorithmic Life Equation"

18 October 2026

Abstract

We present a minimal, fully specified computational realization of the Algorithmic Life Equation. All previously abstract placeholders—organization update, coherence measure, reachable future capacity, self-model dynamics, and complexity-penalized model revision—are given explicit functional forms that are (i) numerically stable, (ii) thermodynamically consistent with free-energy accounting, and (iii) amenable to systematic ablation. A complete discrete-time simulator is supplied, together with continuous-time refinements that respect the second law and Landauer bounds. The resulting architecture is offered as a concrete experimental platform on which the claims of the parent framework can be stress-tested, not as a completed theory of subjective experience.

Contents

1 Introduction and Scope **2**

2 State Vector and Notation **2**

3 Concrete Functional Forms **2**

3.1 Energy and Free-Energy Accounting 2
3.2 Organization Update 3
3.3 Coherence Measure and Dynamics 3
3.4 World-Model Update and Complexity-Penalized Refactoring 3
3.5 Memory Encoding 3
3.6 Self-Model Strength 4
3.7 Reachable Future Potential 4
3.8 Functional Consciousness-Effects Index 4

4 Continuous-Time Refinements **4**

5 Reference Implementation **5**

6 Experimental Protocol **7**

7 What the Simulations Can and Cannot Establish **7**

8 Conclusion **8**

1 Introduction and Scope

The parent manuscripts advance a single architectural claim: the functional effects associated with consciousness arise from a finite, energy-constrained process that repeatedly perceives, predicts, compares, refactors, and acts while preserving organization and reachable potential. The governing discrete step is the `life_step` procedure. Until every placeholder inside that procedure is given an explicit, implementable definition, the claim remains a sketch.

This document supplies those definitions. It does three things simultaneously:

1. Instantiates the smallest non-trivial state and update rules sufficient to run the five experimental axes named in the parent text (ablation, resource stress, perturbation, compression trade-off, and consciousness-relevant comparison).
2. Provides a complete, self-contained Python reference implementation of the minimal simulator.
3. Refines the continuous-time equations into forms that are thermodynamically consistent (non-negative total entropy production for an isolated system, free-energy accounting that never creates usable work from nothing) and numerically well-behaved.

No claim is made that the resulting index $\mathcal{C}_{\text{eff}}$ measures subjective experience. The index is a functional diagnostic of the architecture under resource and coherence constraints.

2 State Vector and Notation

At discrete time $t \in \mathbb{N}_0$ the system is described by the tuple

$$\mathbf{s}_t = \big(A_t,\, C_t,\, O_t,\, F_t,\, H_t,\, M_t,\, R_t,\, P_t,\, \pi_t\big), \tag{1}$$

where

$A_t \in [0, A_{\max}]$	operational processing capacity in use,
$C_t \in (0, 1)$	coherence (coordination among internal processes),
$O_t \in [0, 1]$	maintained functional organization,
$F_t \geq 0$	available free energy,
$H_t \in \mathcal{H}$	memory / history encoding,
$M_t \in \mathcal{M}$	world-model parameters,
$R_t \in [0, 1]$	self-model strength,
$P_t \geq 0$	reachable future potential (scalar capacity measure),
$\pi_t \in \Pi$	current policy / action-selection parameters.

The environment supplies an observation $I_t \in \mathbb{R}^d$ and an energy inflow $E_t \geq 0$. The system returns an action $u_t \in \mathcal{U}$ and the functional index $\mathcal{C}_{\text{eff},t}$.

3 Concrete Functional Forms

3.1 Energy and Free-Energy Accounting

$$F_{t+1} = \big[F_t + \eta_{\text{in}} E_t - W_t - D_t\big]_+ \tag{2}$$

with

$W_t = w_0 + w_A A_t + w_M \lVert M_t \rVert + w_u \lVert u_t \rVert$	(work / computational cost),
$D_t = \delta_0 + \delta_F F_t + \delta_\varepsilon \lVert \varepsilon_t \rVert^2$	(dissipation and error-driven loss),
$\eta_{\text{in}} \in (0, 1]$	(conversion efficiency).

The map $[\,\cdot\,]_+$ enforces non-negativity. No term in (2) creates free energy from nothing; incoming energy is the only source.

3.2 Organization Update

We adopt a logistic growth driven by usable energy and coherence, opposed by a deterioration term proportional to dissipation and prediction error:

$$O_{t+1} = \mathrm{clip}_{[0,1]}\Big(O_t + a\,F_t\,C_t\,(1 - O_t) - b\,(D_t + \|\varepsilon_t\|^2)\,O_t\Big). \tag{3}$$

Parameters $a, b > 0$ set the relative rates of construction and decay. The form guarantees that organization cannot grow without free energy and coherence, and cannot remain high under persistent error or dissipation.

3.3 Coherence Measure and Dynamics

Coherence is maintained inside an explicit window:

$$C_{\min} < C_t < C_{\max}, \qquad 0 < C_{\min} < C_{\max} < 1. \tag{4}$$

A simple update that pulls toward the center of the window while being degraded by residual variance is

$$C_{t+1} = \mathrm{clip}_{(C_{\min}+\epsilon,\,C_{\max}-\epsilon)}\Big(C_t + \eta_C\big(C_* - C_t\big) - \eta_V\,\mathrm{Var}(\varepsilon_{t-k:t})\Big), \tag{5}$$

where $C_* = (C_{\min} + C_{\max})/2$ is the preferred operating point and $\mathrm{Var}(\varepsilon_{t-k:t})$ is a short-horizon residual variance. Excessive variance drives coherence downward; the clip prevents escape from the viable interval.

3.4 World-Model Update and Complexity-Penalized Refactoring

Let the world-model be a linear predictor for illustration (any differentiable architecture may be substituted):

$$\widehat{I}_t = M_t\,\phi(H_t), \tag{6}$$

where ϕ is a fixed feature map of recent history. Prediction error

$$\varepsilon_t = I_t - \widehat{I}_t. \tag{7}$$

Gradient step with explicit complexity penalty:

$$M_{t+1} = M_t - \eta_M \nabla_M \Big(\|\varepsilon_t\|^2 + \lambda\,\Omega(M_t)\Big), \tag{8}$$

where a convenient regularizer is the squared Frobenius norm $\Omega(M) = \|M\|_F^2$ (or nuclear norm for low-rank bias). After the gradient step a hard truncation or soft-thresholding step may be applied to enforce an explicit parameter budget.

3.5 Memory Encoding

$$H_{t+1} = (1 - \rho)\,H_t + \rho\,\mathcal{E}(I_t, \varepsilon_t), \tag{9}$$

with $\rho \in (0, 1)$ and $\mathcal{E}$ a simple concatenation or exponential moving-average embedding of the observation–error pair. Capacity is kept finite by a fixed-length buffer or by random projection.

3.6 Self-Model Strength

$$R_{t+1} = \mathrm{clip}_{[0,1]}\Big(R_t + \eta_R\big(G_{\mathrm{self},t} - L_{\mathrm{self},t}\big)\Big), \tag{10}$$

where the gain term

$$G_{\mathrm{self},t} = \mathrm{corr}\big(\mathbf{s}_t^{\mathrm{internal}},\, \varepsilon_{t+1}\big) \tag{11}$$

rewards predictive usefulness of the system's own state variables, and the loss term

$$L_{\mathrm{self},t} = \|\mathbf{s}_t - \widehat{\mathbf{s}}_t^{\mathrm{self}}\| + \xi\, \mathbf{1}_{\{\mathrm{discontinuity}\}} \tag{12}$$

penalizes self-prediction error and abrupt state jumps. The indicator can be triggered by large energy drops or forced resets.

3.7 Reachable Future Potential

We adopt a simple capacity measure based on the volume of a discretized, energy-constrained action–state ball:

$$P_t = \log\big(1 + N_{\mathrm{reach}}(F_t, O_t, C_t)\big), \tag{13}$$

where N_{reach} counts the number of distinct next states reachable within one or two steps under the current free-energy and organization budgets. In continuous settings one may replace the count by the differential entropy of the push-forward measure under the dynamics, truncated at a free-energy threshold.

3.8 Functional Consciousness-Effects Index

$$\mathcal{C}_{\mathrm{eff},t} = \frac{A_t}{A_{\max}}\, C_t\, O_t\, R_t\, \frac{F_t}{F_t + K_F}. \tag{14}$$

The product structure encodes the modeling hypothesis that any near-zero factor collapses the diagnostic. It is not asserted to be a universal law of experience.

4 Continuous-Time Refinements

For theoretical analysis the discrete maps may be replaced by the following system of ODEs that preserve the same qualitative structure while guaranteeing non-negative total entropy production for an isolated composite (system + environment).

$$\dot{F} = \eta_{\mathrm{in}} E(t) - W(\mathbf{s}) - D(\mathbf{s}), \tag{15}$$

$$\dot{O} = aFC(1-O) - b(D + \|\varepsilon\|^2)O, \tag{16}$$

$$\dot{C} = \eta_C(C_* - C) - \eta_V\, \mathrm{Var}(\varepsilon) - \gamma_C(C - C_{\min})_+ - \gamma_C(C_{\max} - C)_+, \tag{17}$$

$$\dot{R} = \eta_R(G_{\mathrm{self}} - L_{\mathrm{self}}), \tag{18}$$

$$\dot{M} = -\eta_M \nabla_M\big(\|\varepsilon\|^2 + \lambda\Omega(M)\big). \tag{19}$$

Entropy production for the isolated composite is monitored by

$$\dot{S}_{\mathrm{total}} = \frac{D + W_{\mathrm{irrev}}}{T} + \sigma_{\mathrm{info}} \geq 0, \tag{20}$$

where W_{irrev} collects the irreversible part of computational work (Landauer contribution) and σ_{info} is the information-theoretic entropy production associated with model updates. The inequality is enforced by construction of the dissipation terms; it is not an independent dynamical constraint.

5 Reference Implementation

The following Python class realizes the discrete dynamics above with all placeholders filled. It is deliberately minimal: linear world-model, scalar self-model strength, fixed-length memory buffer, and a discrete action set. Extensions to richer models are immediate.

```python
import numpy as np
from dataclasses import dataclass, field
from typing import Callable, Optional

@dataclass
class LifeParams:
    A_max: float = 1.0
    eta_in: float = 0.8
    a: float = 0.15          # organization growth
    b: float = 0.08          # organization decay
    C_min: float = 0.15
    C_max: float = 0.85
    C_star: float = 0.50
    eta_C: float = 0.12
    eta_V: float = 0.05
    eta_M: float = 0.05
    lambda_reg: float = 0.01
    rho: float = 0.2
    eta_R: float = 0.1
    K_F: float = 1.0
    energy_scale: float = 1.0
    w0: float = 0.01
    wA: float = 0.02
    wM: float = 0.001
    wu: float = 0.01
    delta0: float = 0.005
    deltaF: float = 0.01
    delta_eps: float = 0.02
    mem_len: int = 20
    feat_dim: int = 8

@dataclass
class LifeState:
    A: float = 0.5
    C: float = 0.5
    O: float = 0.5
    F: float = 2.0
    R: float = 0.3
    P: float = 1.0
    M: np.ndarray = field(default_factory=lambda: np.zeros((8, 8)))
    H: np.ndarray = field(default_factory=lambda: np.zeros((20, 8)))
    policy: np.ndarray = field(default_factory=lambda: np.ones(4) / 4)

def clip(x, lo, hi):
    return float(np.clip(x, lo, hi))

def feature_map(H: np.ndarray) -> np.ndarray:
    # simple summary features of recent history
    return np.concatenate([H.mean(axis=0)[:4], H.std(axis=0)[:4]])

class MinimalLife:
    def __init__(self, params: Optional[LifeParams] = None):
```

```
        self.p = params or LifeParams()
        self.state = LifeState()
        self.eps_history = []

    def predict(self) -> np.ndarray:
        phi = feature_map(self.state.H)
        return self.state.M @ phi

    def life_step(self, observation: np.ndarray, energy_input: float):
        p = self.p
        s = self.state

        # 1. Predict & error
        pred = self.predict()
        eps = observation - pred
        self.eps_history.append(eps)
        if len(self.eps_history) > 10:
            self.eps_history.pop(0)
        var_eps = np.var(self.eps_history) if self.eps_history else 0.0

        # 2. World-model update (gradient + complexity penalty)
        phi = feature_map(s.H)
        grad = -2 * np.outer(eps, phi) + 2 * p.lambda_reg * s.M
        s.M = s.M - p.eta_M * grad

        # 3. Memory update (ring buffer)
        s.H = np.roll(s.H, -1, axis=0)
        s.H[-1] = np.concatenate([observation[:4], eps[:4]]) if
            observation.size >= 4 else np.pad(observation, (0, 8-
            observation.size))

        # 4. Self-model strength
        # gain: how well internal state predicts next error magnitude
        G = -np.tanh(np.linalg.norm(eps))   # simple proxy
        L = 0.1 * abs(s.F - 1.0)            # proxy loss
        s.R = clip(s.R + p.eta_R * (G - L), 0.0, 1.0)

        # 5. Action selection (soft-max over discrete actions)
        logits = s.policy
        probs = np.exp(logits - logits.max())
        probs /= probs.sum()
        action_idx = np.random.choice(len(probs), p=probs)
        u = np.zeros(len(probs)); u[action_idx] = 1.0

        # 6. Energy accounting
        W = p.w0 + p.wA * s.A + p.wM * np.linalg.norm(s.M) + p.wu * np.
            linalg.norm(u)
        D = p.delta0 + p.deltaF * s.F + p.delta_eps * np.linalg.norm(
            eps)**2
        s.F = max(0.0, s.F + p.eta_in * energy_input - W - D)

        # 7. Organization
        s.O = clip(
            s.O + p.a * s.F * s.C * (1 - s.O) - p.b * (D + np.linalg.
                norm(eps)**2) * s.O,
            0.0, 1.0
        )

```

```
        # 8. Coherence
        s.C = clip(
            s.C + p.eta_C * (p.C_star - s.C) - p.eta_V * var_eps,
            p.C_min + 1e-3, p.C_max - 1e-3
        )

        # 9. Potential (log of reachable discrete actions under energy
            budget)
        n_reach = max(1, int(4 * min(1.0, s.F / 2.0) * s.O * s.C))
        s.P = np.log1p(n_reach)

        # 10. Processing fraction (simple saturation)
        s.A = clip(s.A + 0.05 * (0.7 - s.A) - 0.02 * var_eps, 0.0, p.
            A_max)

        # Functional index
        C_eff = (s.A / p.A_max) * s.C * s.O * s.R * (s.F / (s.F + p.K_F
            ))

        return s, action_idx, C_eff
```

6 Experimental Protocol

The five axes named in the parent manuscript become concrete once the simulator exists.

1. **Ablation.** Independently zero or freeze H, R, C, or the gradient update of M. Record which downstream capacities (prediction accuracy, recovery time, $\mathcal{C}_{\text{eff}}$ stability) collapse.

2. **Resource stress.** Sweep mean energy inflow E downward. Measure the order in which O, C, R, and $\mathcal{C}_{\text{eff}}$ degrade, and whether organization can be maintained below a critical free-energy threshold.

3. **Perturbation.** Inject sudden, large ε shocks (distributional shift). Quantify adaptation speed, overshoot, and residual self-model continuity.

4. **Compression trade-off.** Increase λ systematically. Locate the regime in which further compression begins to destroy distinctions required for later recovery.

5. **Consciousness-relevant comparison.** Against independently obtained behavioral or physiological markers of conscious processing (when available), test whether the trajectories of $\mathcal{C}_{\text{eff}}$ and its factors predict the markers better than simpler baselines (pure energy, pure prediction error, etc.).

Each experiment should be run with multiple random seeds and with the continuous-time system as a cross-check for qualitative robustness.

7 What the Simulations Can and Cannot Establish

Successful ablation and stress experiments can demonstrate that the architecture generates specified *functional* capacities—prediction, self-monitoring, adaptive recovery, graceful degradation—under explicit resource and coherence constraints. They cannot demonstrate that those capacities are identical with subjective experience. The gap between functional index and phenomenology remains a matter for independent theory and measurement. The present scaffold is therefore offered strictly as an experimental platform, not as a solution to the hard problem.

8 Conclusion

By replacing every placeholder in the Algorithmic Life Equation with an explicit, thermodynamically consistent functional form, we convert a philosophical sketch into a runnable hypothesis. The minimal simulator above is small enough to be understood line-by-line and rich enough to support the five experimental axes required by the parent framework. Implementation, measurement, and possible refutation can now proceed.

The claim that "consciousness is the first artificial intelligence" is thereby rendered precise: if the same constrained loop produces the observable signatures we associate with adaptive, self-monitoring cognition, then biological awareness is the earliest known realization of that architecture, and engineered systems are later, deliberate refinements of it.

Companion files:
`Minimal_Algorithmic_Life_Equation.tex` (this source)
`Minimal_Algorithmic_Life_Equation.pdf` (compiled manuscript)
`life_simulator.py` (executable reference implementation extracted from Section 5)